

**SCHOOL OF
COMPUTING**

Design Analysis of Algorithm

CSE211

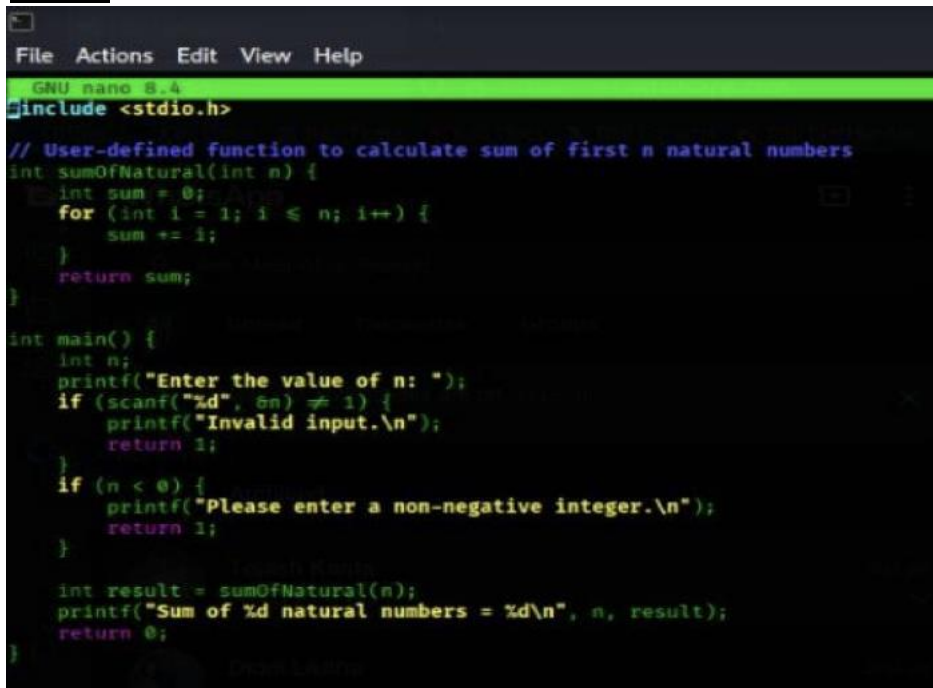
Week-1

CH.SC.U4CSE24221

TEJESH KANTA

1. Write a program to find sum of first n natural numbers using user defined function.

Code:



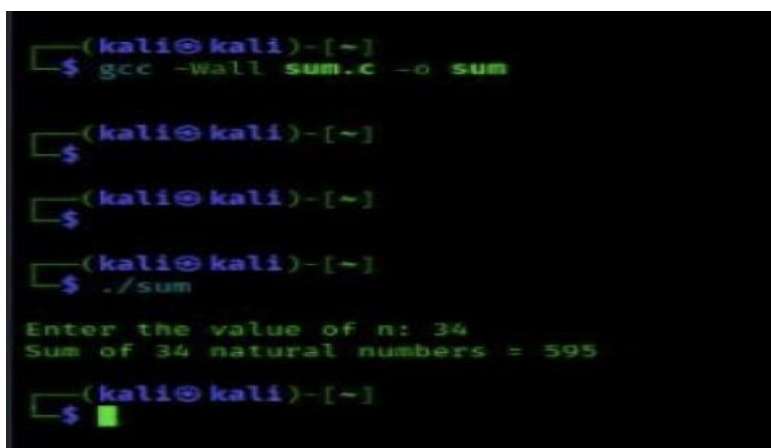
```
File Actions Edit View Help
GNU nano 2.9.4
#include <stdio.h>

// User-defined function to calculate sum of first n natural numbers
int sumOfNatural(int n) {
    int sum = 0;
    for (int i = 1; i <= n; i++) {
        sum += i;
    }
    return sum;
}

int main() {
    int n;
    printf("Enter the value of n: ");
    if (scanf("%d", &n) != 1) {
        printf("Invalid input.\n");
        return 1;
    }
    if (n < 0) {
        printf("Please enter a non-negative integer.\n");
        return 1;
    }

    int result = sumOfNatural(n);
    printf("Sum of %d natural numbers = %d\n", n, result);
    return 0;
}
```

Output:



```
(kali@kali)-[~]
$ gcc -Wall sum.c -o sum

(kali@kali)-[~]
$

(kali@kali)-[~]
$

(kali@kali)-[~]
$ ./sum
Enter the value of n: 34
Sum of 34 natural numbers = 595

(kali@kali)-[~]
$
```

Space Complexity and it's justification:

- Space Complexity: $O(1)$ because only a fixed number of variables are used.
- In main(), there is only **one int variable** (n).
- In sum_n(), only **two int variables** (s and i) are used.
- No arrays or dynamic memory → space stays constant.

2. Write a program to find sum of squares of first n natural numbers

Code:

```
GNU nano 8.4
#include <stdio.h>

int main() {
    int n, i;
    int sum = 0;

    printf("Enter a number: ");
    scanf("%d", &n);

    for(i = 1; i <= n; i++) {
        sum += i * i; // adding square of each number
    }

    printf("Sum of squares of first %d natural numbers = %d\n", n, sum);
    return 0;
}
```

Output:

```
(kali@kali)-[~]
$ ./sum_square

Enter a number: 2
Sum of squares of first 2 natural numbers = 5
Enter the value of n:
(kali@kali)-[~]
$
```

Space Complexity and it's justification:

- Space Complexity: $O(1)$
- main() uses only one int (n).
- sum_sq() uses two ints (s, i).
- No arrays → constant memory.

3. Write a program to find sum of cubes of first n natural numbers

Code:

```
GNU nano 8.4
#include <stdio.h>

int main() {
    int n, i;
    long long sum = 0; // to store big values safely

    printf("Enter the value of n: ");
    scanf("%d", &n);

    // loop from 1 to n
    for(i = 1; i <= n; i++) {
        sum = sum + (long long)i * i * i; // add cube of i
    }

    printf("Sum of cubes of first %d natural numbers = %lld\n", n, sum);

    return 0;
}
```

Output:

```
(kali@kali)~[~]
$ nano sum_cubes.c

(kali@kali)~[~]
$ gcc sum_cubes.c -o sum_cubes

(kali@kali)~[~]
$ ./sum_cubes

Enter the value of n: 2
Sum of cubes of first 2 natural numbers = 9

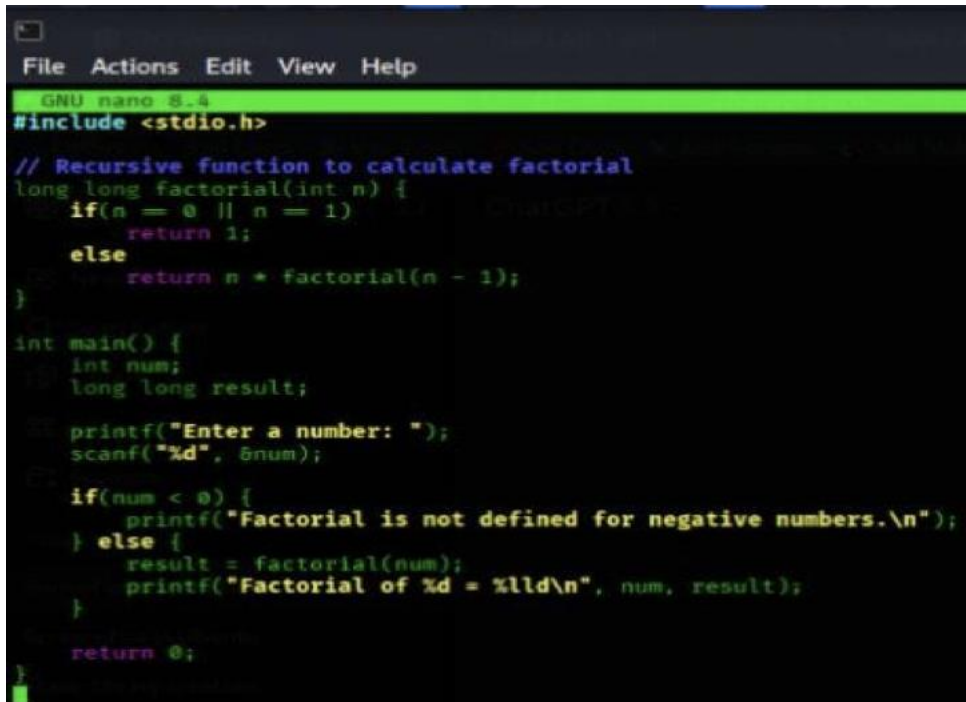
(kali@kali)~[~]
$
```

Space Complexity and it's justification:

- Space Complexity: $O(1)$
- Only one int in main().
- Only two or three ints inside the function.
- Memory stays constant.

4. Write a program to find factorial of a given integer using recursion.

Code:



```
GNU nano 8.4
#include <stdio.h>

// Recursive function to calculate factorial
long long factorial(int n) {
    if(n == 0 || n == 1)
        return 1;
    else
        return n * factorial(n - 1);
}

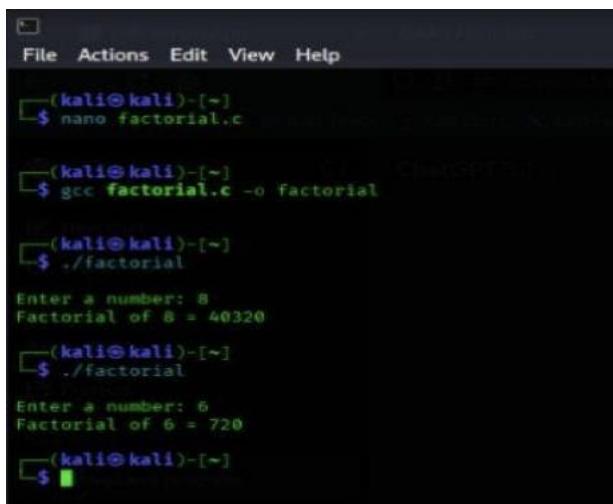
int main() {
    int num;
    long long result;

    printf("Enter a number: ");
    scanf("%d", &num);

    if(num < 0) {
        printf("Factorial is not defined for negative numbers.\n");
    } else {
        result = factorial(num);
        printf("Factorial of %d = %lld\n", num, result);
    }

    return 0;
}
```

Output:



```
(kali@kali)-[~]
$ nano factorial.c

(kali@kali)-[~]
$ gcc factorial.c -o factorial

(kali@kali)-[~]
$ ./factorial

Enter a number: 8
Factorial of 8 = 40320

(kali@kali)-[~]
$ ./factorial

Enter a number: 6
Factorial of 6 = 720

(kali@kali)-[~]
$
```

Space Complexity and it's justification:

- ✓ Space Complexity: $O(n)$
- ✓ Each recursive call adds one frame to the stack.
- ✓ Maximum depth is n .
- ✓ So space grows linearly with input.

5. Write a program to transpose a 3x3 matrix.

Code:

```
GNU nano 8.4
#include <stdio.h>

int main() {
    int matrix[3][3], transpose[3][3];
    int i, j;

    // Input elements for 3x3 matrix
    printf("Enter 9 elements of the matrix:\n");
    for(i = 0; i < 3; i++) {
        for(j = 0; j < 3; j++) {
            scanf("%d", &matrix[i][j]);
        }
    }

    // Finding transpose
    for(i = 0; i < 3; i++) {
        for(j = 0; j < 3; j++) {
            transpose[j][i] = matrix[i][j];
        }
    }

    // Printing original matrix
    printf("\nOriginal Matrix:\n");
    for(i = 0; i < 3; i++) {
        for(j = 0; j < 3; j++) {
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }

    // Printing transpose matrix
    printf("\nTranspose of the Matrix:\n");
    for(i = 0; i < 3; i++) {
        for(j = 0; j < 3; j++) {
            printf("%d ", transpose[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

Output:

```
(kali@kali)-[~]
$ ./transpose.c
Enter 9 elements of the matrix:
1
2
3
4
5
6
7
8
9

Original Matrix:
1 2 3
4 5 6
7 8 9

Transpose of the Matrix:
1 4 7
2 5 8
3 6 9
```

Space Complexity and it's justification:

- Space Complexity: $O(1)$ (because matrix size is fixed 3×3)

6. Write a program to find fibonacci series

Code:

```
File Actions Edit View Help
GNU nano 8.4
#include <stdio.h>

int main() {
    int n, i;
    int a = 0, b = 1, next;

    printf("Enter how many terms you want: ");
    scanf("%d", &n);

    printf("Fibonacci Series: ");

    for(i = 1; i <= n; i++) {
        printf("%d ", a);
        next = a + b;
        a = b;
        b = next;
    }

    printf("\n");
    return 0;
}
```

Output:

```
File Actions Edit View Help
(kali@kali)-[~]
$ nano fibonacci.c

(kali@kali)-[~]
$ gcc fibonacci.c -o fibonacci

(kali@kali)-[~]
$ ./fibonacci
Enter how many terms you want: 5
Fibonacci Series: 0 1 1 2 3
$
```

Space Complexity and it's justification:

- Space Complexity: $O(1)$
- Uses only a few ints (a, b, c, i, n).
- No arrays or recursion.
- Memory fixed $\Rightarrow$ constant space.